Protocol Impersonation Detection System

Dylan Hartley - CECS 478

Problem Statement

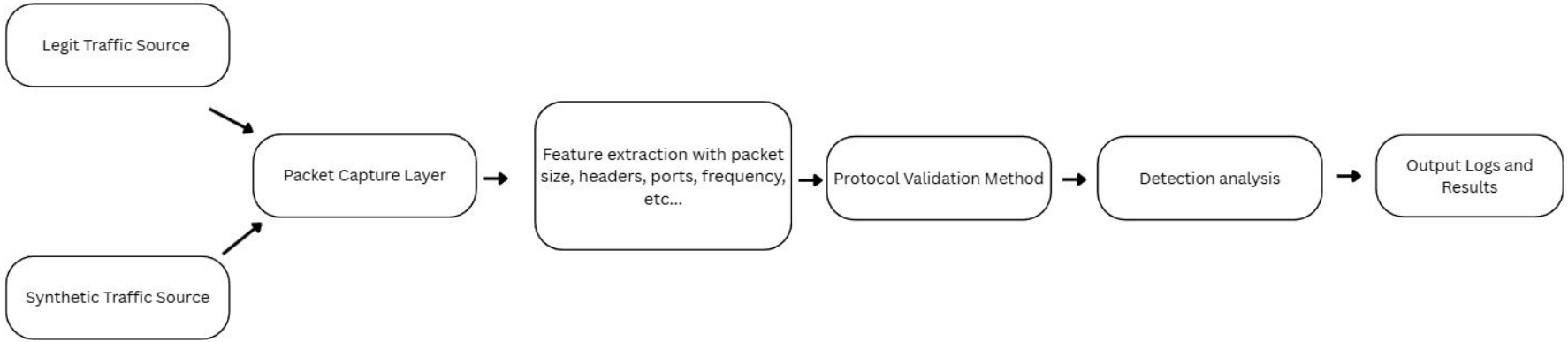
The most common sign of malicious behavior over a network is an attacker sending high volumes of traffic to the network

One attack signature that could be harder to detect are attacks that serve to mimic the actions and structure of legit application layer protocols like HTTP/HTTPS, or DNS.

This project addresses this issue by developing a simplified system that will detect these protocol impersonation attacks,

System Design

This protocol impersonation detection system is an application that consists of a pipeline of six modules that perform in sequence of one another



Threat Model

The Assets:

- Any network service that connects using application layer protocols

The Attacker Capabilities:

- The attacker in this scenario is the person that is sending these malicious packets over the network
- In this case the malicious packets in this case are synthetically generated by the program

Assumptions:

- It is assumed that all the traffic is captured on a local network and no external traffic is there
- It also assumes that the attacker cant modify any of the validators source code or the docker container used
- It also assumes that the payloads of these packets are not encrypted and are only in plaintext

Risks not counted for:

- It does not model encrypted TLS inspection and the model does not account for any live traffic

Types of Malicious Packets

Request Smuggling Indicator:

- Duplicate Content-Length headers

Null Byte Injection:

- Binary/null data hidden after valid-looking HTTP headers

Malformed DNS Queries:

- Truncated DNS headers or invalid query structure

DNS Tunneling Pattern:

- Long encoded-looking DNS labels used to hide data

DNS Response Masquerading:

- DNS response packets pretending to be queries using the QR bit

HTTP on DNS Port:

- HTTP request text sent to UDP port 53 to imitate DNS traffic

DNS on HTTP Port:

- DNS binary payloads sent to TCP port 80 to imitate HTTP traffic

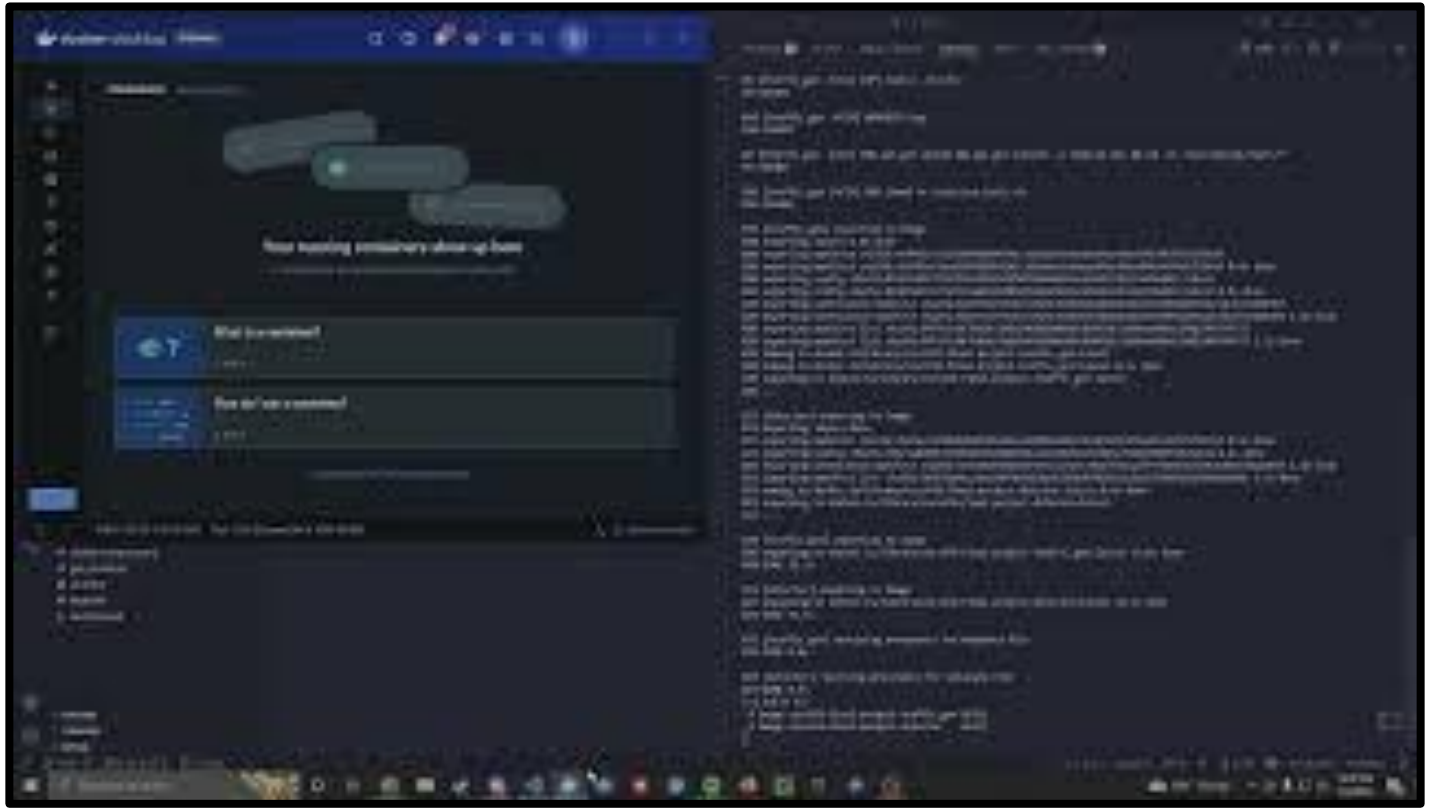
Malformed HTTP Requests:

- Invalid methods, missing Host headers, or missing HTTP versions

HTTP Version Spoofing:

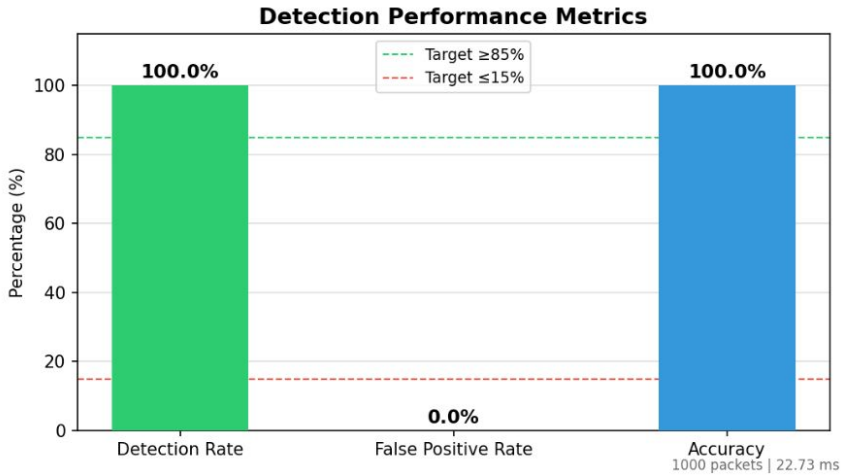
- Unsupported versions like HTTP/1.9 that look close to valid HTTP

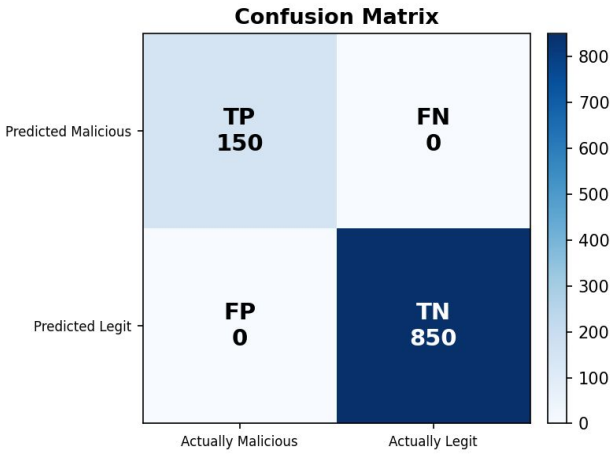
Demo



Results

The detector beat the goal of 85% detection rate by classifying the malicious packets at a rate of 100%. It also displays the number of packets (1000) and the response time (22 ms)





Limitations

The dataset used is synthetic. Since the pcap file is made for easy reproducibility, it doesn't capture the full details of a real world network

The detector methods and model uses very deterministic rules. This was to make sure that the detector processed the information quickly. This means that it can possibly miss attacks that are outside the scope of the definitions

This system is set up to analyze packets that have already been captured and does not work on real time networks. It does not run inline like a firewall or block traffic in real time

Future Work/Features

Test against real captured traffic with tools like curl, dig, nslookup, and browser requests. This would make the evaluation closer to real world data and would take a more realistic approach to detection systems

Parse the HTTP and DNS at a deeper level. This would help handle more detailed HTTP syntax, additional DNS records, and have a clearer separation for HTTPS/TLS

Analyze packets at a connection level. As of now, the packets are classified independently of one another. If we were able to track the flow of the packet, it could reveal repeated malicious requests or filter behaviors through different layers

Thank You!

Dylan Hartley - CECS 478